

Software Engineering 2

1 Einführung Software-Engineering-2

Prof. Dr.-Ing. Sven Graupner



Gliederung

1 Begriffe (Wiederholung)

2 Was macht ein Software Engineer?

2.1 Software Design Engineer (SDE)

2.2 Software QA Engineer

2.3 Full-Stack Software Engineer

2.4 DevOps Engineer

2.5 Software Architect

3 Software Delivery (SD)

4 Continuous Software Delivery (CSD)

5 Continuous Integration (CI)

6 Continuous Deployment (CD)

7 DevOps

1 Begriffe (Wiederholung)



Software Engineering: ist die zielorientierte Bereitstellung und systematische Verwendung von Prinzipien, Methoden und Werkzeugen für die *arbeitsteilige, ingenieurmäßige Entwicklung und Anwendung* von umfangreichen *Softwaresystemen*.

Balzert, H.: Lehrbuch der Software-Technik. Spektrum Akademischer Verlag, ISBN 3-8274-0480-0, Heidelberg 1996, 1998, 2001.



1.1 Software Engineering ::= Software + Engineering

Software: ist Gegenstand des Software Engineering. Software umfasst die Gesamtheit der Information zur Steuerung und Verarbeitung in programmierbaren Rechensystemen. Software umfasst Programme und Daten, aber auch Dokumentation.

Engineering: lat. *ingenium* (Klugheit) und *ingeniare* (ausdenken, entwickeln, bewerkstelligen) bezeichnet die strukturierte, arbeitsteilige und auf Grundlage anerkannter Praktiken basierende Vorgehensweise zur zielgerichteten *Herstellung komplexer Konstruktionen*, wie Bauwerke, Maschinen oder Softwaresysteme.

Im Unterschied zu Wissenschaft, lat. *scientia*, die Wissen systematisch sammelt, ordnet und erweitert, um Phänomene aus Natur oder Gesellschaft zu erklären (aber nicht konstruiert oder baut), geht es im Software Engineering um die Herstellung von Softwaresystemen.

1.2 Einordnung des Software Engineerings

- Die Grundlagen des Software Engineerings aus SE-1 waren:
 - „Programmierung“ wird in einen Kontext eingeordnet (was?, für wen? wer? wo?...)
 - Planung (Vorausdenken, Abstimmung mit Beteiligten):
 - des Produkts (-> Software) -> aber eig. Dienste der Software (d.h. der Betrieb der Software).
 - der Prozesse (Planung, Herstellung, Auslieferung, Betrieb, Wartung/Upgrades/Updates).
 - Durchführung der Vorgänge:
 - Produkt-Management (Erwartungen/Anforderungen, Ideen/Features ...) -> Product Roadmap.
 - Projekt-Management (Ressourcen: Zeit, Budget, Teams, ...)
 - Management der Entwicklung (der Architektur, der Codebase, der Auslieferung ...).

1.2.1 Worum geht es im Software Engineering?

Software
als **Produkt**
(Software, Service)

Definition/Planung des Produkts selbst:

- Was ist das Produkt?
- Wer sind Nutzer, Zielgruppen?
- Wieviel?
- Wie ist die Nutzung?
- Wann? Wie lange?
- Preis? Qualität?

Software
als **Prozess**
(der Herstellung)

Definition/Planung und Ausführung der Herstellung:

- Wer macht was?
- Was, wann und wieviel braucht man? Wo?
- Wann? Wie lange?
- Welche Kompetenzen?
- Rollen, Verantwortlichkeiten.
- Preis? Qualität?

Software
als **System**
(im Betrieb)

Definition/Planung und Ausführung des Betriebs:

- Wer betreibt? Was? Wo?
- Wieviel? Wann? Wie lange?
- Für wen?
- Wer ist zuständig bei Fehlern oder Ausfall?
- Preis? Qualität?

1.3 Kernprozesse der Software Herstellung

Im Fokus steht die **Herstellung der Software**.

1. Anforderungen	2. Analyse SE-1	3. Entwurf SE-2	4. Entwicklung	5. Qualitätssicherung
Problemdefinition – Was? Anforderungserhebung. - Stakeholder, Akteure, Rollen, Szenarien, - Use Cases, User Stories, Glossar. Anforderungsanalyse. - Ableitung mgl. voll-ständ. u. konsistenter Systemanforderungen, - fkt. (FR) u. nicht-funkt. Anforderungen (NFR). <u>Lastenheft</u> als Spezifikation aus Auftraggebersicht. Lösungsvorschlag - Bewertung. Auftragsvergabe. Vorgehensweise - Linear, iterativ, agil.	Spezifikation eines SW-Systems mit: Konzepten u. Strukturen: - Domänen, - Konzept-Klassen, - Relationen, - Kardinalitäten. äußeres Verhalten („black-box“-Modell) u. prinzipieller innere Arbeitsweise („white-Box“-Modell) mit: - Prozessen, - Zustandsautomaten - Interaktionen. <u>Pflichtenheft</u> mit Spezifikationen der: - der zu entw. Software. - Abnahmekriterien.	Umsetzung der Spezifikation in Entwürfe . Definition der: Logischen Architektur (Geschäftslogik) mit - Log. Komponenten, - Geschäftsobjekten, - Geschäftsprozessen. Technischen Architektur - Zuordnung der log. Elem. zu techn. Komponenten. - Entwürfe für Bedienung und UI. - Festlegung des Testplans. - Festlegung der Abnahmekriterien.	Umsetzung der Entwürfe - in Code. - in techn. Dokumentation. Validierung mittels - Tests (Unit-, Integrations-, Akzeptanz-, Performance-) - Reviews, Abnahme, Rel. Feature Management - Product backlog. Konfig. und Release Mgmt. - Versionskontrolle (git). - Branch-Management Artefakt Management - Code, Data, Libs, Docs. Code Quality Mgmt CQM - Metriken, Werkzeuge.	Stetige Umsetzung der Qualitätsanforderungen. Testpläne erstellen und ausführen. Akzeptanztests f. Releases. - Reviews und Abnahmen. - Herstellung und Betrieb der Testumgebung(en). Testautomatisierung. Code Quality Mgmt CQM - Metriken definieren, - Werkzeuge einsetzen. Defect Management. - Erfassung, Priorisierung. Customer Support - Incident Management, - Problem Manag.

1.4 Unterstützungsprozesse

Im Fokus steht die **Organisation der Herstellungsprozesse** der Software.

Organisations- und Prozeß-Management	Projekt-Management	Produkt-Management	Produkt-Marketing	Qualitäts-Management
• Festlegung von Zielen: was, wann und warum? • Koordination der Stakeholder. • Ressourcenmanagement. • Herstellen von Strukturen u. Verantwortlichkeiten. • Projektdefinition. • Prozess-Definitionen. • Verbesserungs-Mgmt.	Planung, Organisation, Steuerung eines Projekts mit: • Zielen, Zeiten, Etappen. • Ressourcen-Mgmt. • Hiring, Festlegen von Verantwortlichkeiten. • Risiken bewerten. • Monitoring, Reporting des Projektfortschritts. Org. der Rahmenbed.: • Dev-, Test-, Produktions-Umgebungen.	Planung, Steuerung und Kontrolle des Produkts mit Verantwortung für: • Beziehungen zu Stakeholdern. • Anforderungs-Management. • Release-Planung und Steuerung (Product Road Map). • Qualitätsmanagement. • Risikomanagement. • Integration des Produkts in das Unternehmensangebot.	Promotion und Verkauf. • Marktanalyse, Identifikation von Zielgruppen. • Formulierung einer der Zielgruppe verständlichen Nachricht (messaging). • Vertretung des Produkts bei Kunden und in der Öffentlichkeit. • Herstellung und Vertrieb von Werbe-Material. • Umfragen, Auswertung. • Online-Marketing.	Qualitätskriterien festlegen und wie sie überprüft werden: • für das Produkt und • Herstellungsprozess. Organisation der Reaktion auf Q-Mängel: • Incident-Mgmt, • Problem-Mgmt. • Support. Dokumentation, Review, Abnahme der QM Prozesse.

1.5 Rollen in der Software-Entwicklung

Organisatorische Rollen

Organisations- und Prozeß-Management	Projekt-Management	Produkt-Management	Produkt-Marketing	Qualitäts-Management
Director Program Manager	Project Manager Scrum Master	Product Manager (PM) Product Owner (PO)	Product Marketing Role (Pre-) Sales Engineer	Customer Support

Technische Rollen

1. Planung	2. Analyse	3. Entwurf	4. Entwicklung	5. Qualitätssicherung
	Requirements Engineer	Solution Architect Software Architect UI/UX Designer	Software Developer Release Engineer DevOps Engineer	Quality Assurance (QA) Engineer

2 Was macht ein Software Engineer?



Der Software Engineer kennt alle Phasen des Software Lebenszyklus und wendet dort ingenieur-gemäße Techniken an:

- zur Formulierungen und Kommunikation von **Anforderungen** (was wird entwickelt?),
- **Software-Design**: Schaffung von Strukturen in Daten und Code, deren Kommunikation über Zeichnungen (UML-Diagramme); die Spezifikation (= exakte Definition) von Software-Komponenten durch Schnittstellen und Protokolle (=Interaktionen zwischen Komponenten).
- **Implementierung** - Entwicklung des Codes sowie der Unit Tests.
- Management der Entwicklung (z.B. branches in git; aber auch Feature-, Bug-, Issue-Mgmt ...).
- **Test/QA**: Entwicklung, Aufbau und Durchführung von Tests sowie automatisierter Test-Systeme für komplexe Tests wie System-, Integrations-, Performance-, Usability-Tests.
- **Release, Deployment** (=Installation in Zielumgebung), **Betrieb** (Ops) der Software.

2.1 Software Design Engineer (SDE)



Software Design Engineer (SDE) ist eine Einstiegsposition in Firmen. Es wird erwartet, dass der SDE gegebene Software-Strukturen (Datenstrukturen, Architekturen, Schnittstellen, usw.) in funktionsfähigen Code umsetzen kann.

- **Software-Design:** Schaffung von Strukturen in Daten und Code, deren Kommunikation über Zeichnungen (UML-Diagramme); die Spezifikation (= exakte Definition) von Software-Komponenten durch Schnittstellen und Protokolle (=Interaktionen zwischen Komponenten).
- **Implementierung** - Entwicklung des Codes sowie der Unit Tests.
- Management der Entwicklung (z.B. branches in git; aber auch Feature-, Bug-, Issue-Mgmt ...).
- **Test/QA:** Entwicklung, Aufbau und Durchführung von Tests sowie automatisierter Test-Systeme für komplexe Tests wie System-, Integrations-, Performance-, Usability-Tests.
- **Release, Deployment** (=Installation in Zielumgebung), **Betrieb** (Ops) der Software.

2.2 Software QA Engineer



Ein **Quality Assurance (QA) Engineer** ist auf den Test von Software spezialisiert. Zu den Aufgaben gehören u.a. die Entwicklung, Aufbau und Durchführung von Tests sowie automatisierter Test-Systeme für komplexe Tests wie System-, Integrations-, Performance-, Usability-Tests.

- **Software-Design:** Schaffung von Strukturen in Daten und Code, deren Kommunikation über Zeichnungen (UML-Diagramme); die Spezifikation (= exakte Definition) von Software-Komponenten durch Schnittstellen und Protokolle (=Interaktionen zwischen Komponenten).
- **Implementierung** - Entwicklung des Codes sowie der Unit Tests.
- Management der Entwicklung (z.B. branches in git; aber auch Feature-, Bug-, Issue-Mgmt ...).
- **Test/QA:** Entwicklung, Aufbau und Durchführung von Tests sowie automatisierter Test-Systeme für komplexe Tests wie System-, Integrations-, Performance-, Usability-Tests.
- **Release, Deployment** (=Installation in Zielumgebung), **Betrieb** (Ops) der Software.

2.3 Full-Stack Software Engineer



Ein **Full-Stack SW Engineer** kennt den gesamten Lebenszyklus („Stack“) von Software, von der Entwicklung (Dev), über Tests, die Auslieferung bis zum Betrieb (Ops) von Software, sowie den Bau und Betrieb automatisierter **DevOps** Pipelines.

- **Software-Design:** Schaffung von Strukturen in Daten und Code, deren Kommunikation über Zeichnungen (UML-Diagramme); die Spezifikation (= exakte Definition) von Software-Komponenten durch Schnittstellen und Protokolle (=Interaktionen zwischen Komponenten).
- **Implementierung** - Entwicklung des Codes sowie der Unit Tests.
- Management der Entwicklung (z.B. branches in git; aber auch Feature-, Bug-, Issue-Mgmt ...).

- **Test/QA:** Entwicklung, Aufbau und Durchführung von Tests sowie automatisierter Test-Systeme für komplexe Tests wie System-, Integrations-, Performance-, Usability-Tests.
- **Release, Deployment** (=Installation in Zielumgebung), **Betrieb** (Ops) der Software.

2.4 DevOps Engineer



Der **DevOps** Engineer ist auf den Bau und Betrieb der automatisierter **DevOps Pipelines** spezialisiert. Er befasst sich primär mit Release, dem automatisierten Test sowie der automatisierten Auslieferung (delivery) der Software in ein Produktivsystem.

- **Software-Design:** Schaffung von Strukturen in Daten und Code, deren Kommunikation über Zeichnungen (UML-Diagramme); die Spezifikation (= exakte Definition) von Software-Komponenten durch Schnittstellen und Protokolle (=Interaktionen zwischen Komponenten).
- **Implementierung** - Entwicklung des Codes sowie der Unit Tests.
- Management der Entwicklung (z.B. branches in git; aber auch Feature-, Bug-, Issue-Mgmt ...).
- **Test/QA:** Entwicklung, Aufbau und Durchführung von Tests sowie automatisierter Test-Systeme für komplexe Tests wie System-, Integrations-, Performance-, Usability-Tests.
- **Release, Deployment** (=Installation in Zielumgebung), **Betrieb** (Ops) der Software.

2.5 Software Architect



Ein **Software Architect** ist für die technische Umsetzung der Software verantwortlich.

Er legt u.a. die Strukturen fest, wie Datenmodelle, Komponenten, Interfaces, Protokolle.

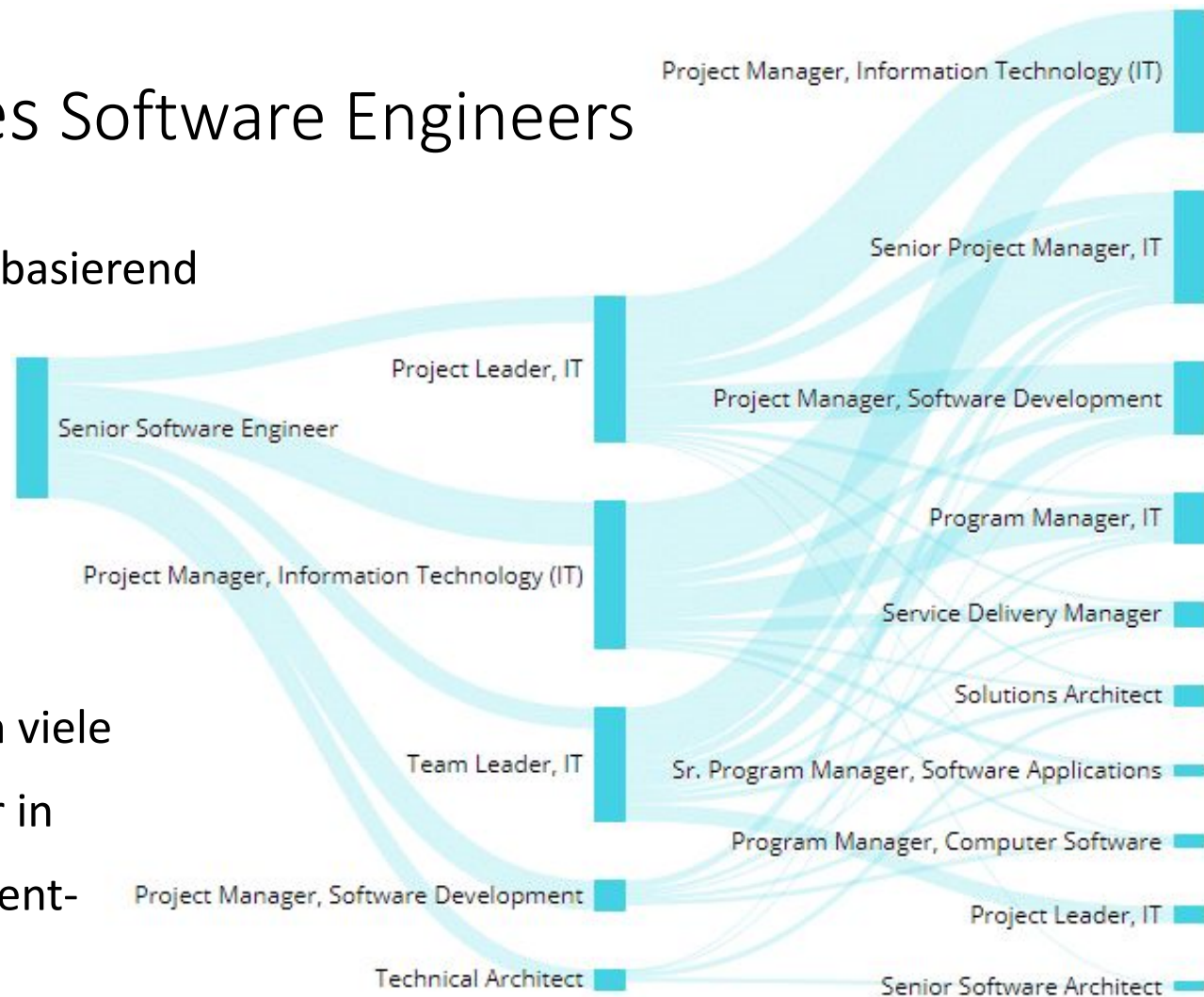
- **Software-Design:** Schaffung von Strukturen in Daten und Code, deren Kommunikation über Zeichnungen (UML-Diagramme); die Spezifikation (= exakte Definition) von Software-Komponenten durch Schnittstellen und Protokolle (=Interaktionen zwischen Komponenten).
- **Implementierung** - Entwicklung des Codes sowie der Unit Tests.
- Management der Entwicklung (z.B. branches in git; aber auch Feature-, Bug-, Issue-Mgmt ...).
- **Test/QA:** Entwicklung, Aufbau und Durchführung von Tests sowie automatisierter Test-Systeme für komplexe Tests wie System-, Integrations-, Performance-, Usability-Tests.
- **Release, Deployment** (=Installation in Zielumgebung), **Betrieb** (Ops) der Software.

2.7 Karrieren eines Software Engineers

Die Grafik zeigt Karrierewege basierend auf Daten von LinkedIn.

Software Engineer ist eine gute Einstiegsposition nach dem Studium.

Nach ein paar Jahren hat man viele Optionen, sich technisch oder in anderen Bereichen weiter zu entwickeln.



3 Software Delivery (SD)

Als **Software Delivery (SD)** bezeichnet man den gesamten Prozess der Herstellung, Verfügbarmachung bis zur Nutzung einer Software oder eines neuen Software Features.

- Der Prozess beginnt mit der „Idee“ (Anforderung, Wunsch) und endet, wenn die Software in Nutzung (nicht nur benutzbar) ist.



5 Jahre bei klassischen IT-/Software-Upgrades (z.B. SAP).

1.5 Jahre bei schnellen Upgrade Zyklen.

Was passiert entlang dieses Prozesses?

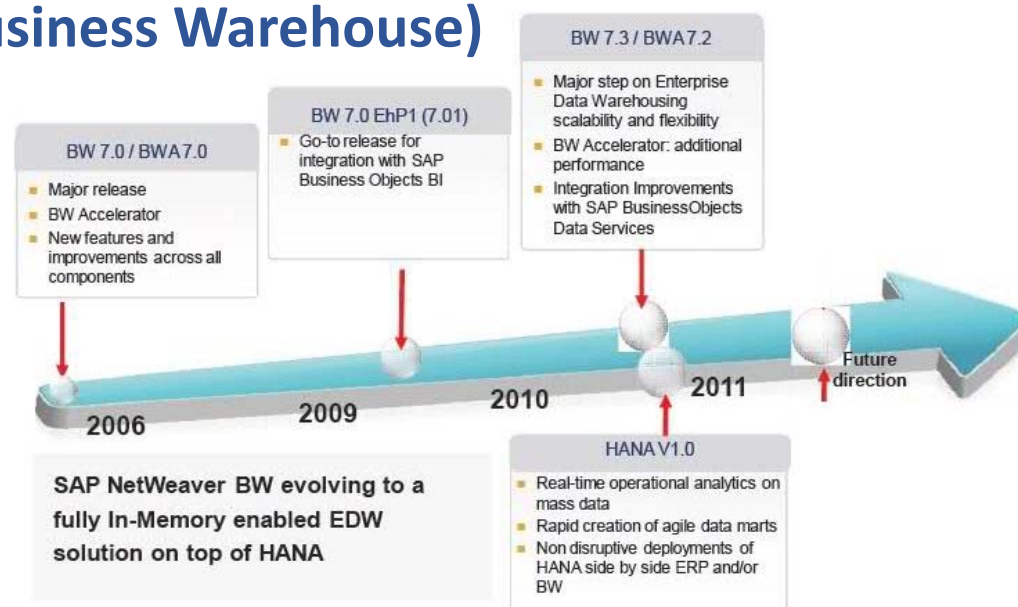


3.1 Software Delivery bei „klassischer Software“

Wie lange dauert es, bis die Idee für ein neues Feature beim Software-Hersteller (Developer) in Software umgesetzt ist?

Beispiel SAP BW (Business Warehouse)

- Version: v7.0 -> v7.01
= **3.5 Jahre**
(2006-2009)
- v7.4 -> 7.5
= 23 Monate.



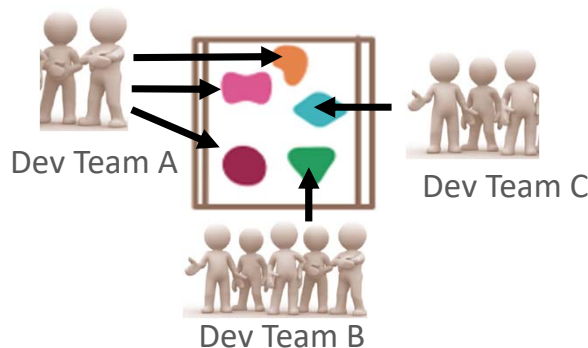
Name	Version	Release Datum
BIW	1.0E	1997
BIW	1.2A	Oktober 1998
BIW	1.2B	September 1999
BIW	2.0A	Februar 2000
BIW	2.0B	Juni 2000
BIW	2.1C	November 2000
BW	3.0A	Oktober 2001
BW	3.0B	Mai 2002
BW	3.1	November 2002
BW	3.1C	Mai 2003
BW	3.3	Oktober 2003
BW	3.5	April 2004
BI	7	Juli 2005
BW	7.3	Juli 2011
BW	7.4	Dezember 2013
BW	7.5	Oktober 2015

https://de.wikipedia.org/wiki/SAP_NetWeaver_Business_Intelligence

3.2 Traditionelle Fragmentierung der SD

Software-Delivery ist in verschiedene Teams, teils über verschiedene Firmen aufgespaltet, zwischen denen Übergaben erfolgen müssen.

Software-Entwicklung



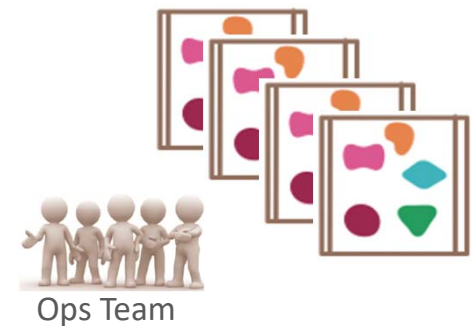
Integration und Test



CERTIFIED PARTNER



Betrieb



Hewlett Packard
Enterprise

4 Continuous Software Delivery (CSD)

Als **Continuous (Software) Delivery (CD)** bezeichnet man den Prozess der („schnellen“) kontinuierlichen Herstellung und Verfügbarmachung neuer Software Features.

- Der Prozess beginnt mit der „Idee“ (Anforderung, Wunsch) und endet, wenn diese in der Software benutzbar ist.



4.1 Phasen der Software Delivery (SD)

1. **Idee**/Anforderungen sammeln, und z.B. als User Stories formulieren, priorisieren (z.B. in Product Backlog bei Scrum) und für Sprint ausw.
2. Im Sprint User Stories umsetzen und **Feat. entwickeln** (Programmierung).
3. **Unit Tests** der eig. Entwicklung.
4. „**Pull Request**“ – Aufforderung zu Review und Freigabe der Entwicklung.
5. Zusammenführen (**Integration**) mit anderen Entwicklungen (git merge).
6. Unit Tests. **Integrations-Tests**.
7. Abzweigen aller fertigen Entwicklungen als **Release-Kandidat**.

4.2 SD, Phase 2: – Test, Release („Dev“)

8. **Test des Release-Kandidaten** auf sep. Test-Systemen:

- Installation („Deployment“) in eine Test-Umgebung, d.h. auf spezielle Server-Maschinen, um umfassende Tests vor der Freigabe auszuführen:
 - Anschluss realistischer Daten(-Banken), Netzwerk-Verb., Nutzer-/Rechte, Filesysteme, usw.
- Umfassende Tests des Release-Kandidaten: Funktions-System-, Betriebs-, Performance-Tests.

9. **Release**, d.h. Freigabe des Release-Kandidaten.

Die Release beendet die Entwicklungs- oder „Dev“-Phase.

Release Party



4.3 SD, Phase 3: – Release to Production („Ops“)

10. **Test der Release** durch das Betriebs-**“Ops“-Team**, welches die Software für die eigentliche Nutzung „in Production“ betreibt.

- Installation („Deployment“) in eine **Staging Umgebung**, d.h. auf Server, welche dem Produktivsystem ähnlich sind (gleiche Hardware, Betriebssystem usw.).
 - Anschluss realistischer Daten(-Banken), Netzwerk-Verb., Nutzer-/Rechte, Filesysteme, usw.
- erneute umfassende Tests des Release-Kandidaten in der Staging Umgebung unter realist. Bedingungen: Funktions-, System-, Betriebs-, Performance-Tests.

11. **„Release to Production“ („RP“)**, d.h. die Freigabe der neuen Release für das Produktivsystem.

4.4 SD, Phase 4: – Deployment to Production („Ops“)

12. **Deployment** in das/die Produktivsystem(e), d.h. die Installation der neuen Software Release im Produktivsystem.

- Konfiguration, Anbindung an die realen Daten, usw.

13. **Inbetriebnahme („Roll-Out“)** der Release in Production, d.h. das vollst. oder teilweise Umleiten von Nutzeranfragen auf die neue Release im Produktivsystem.

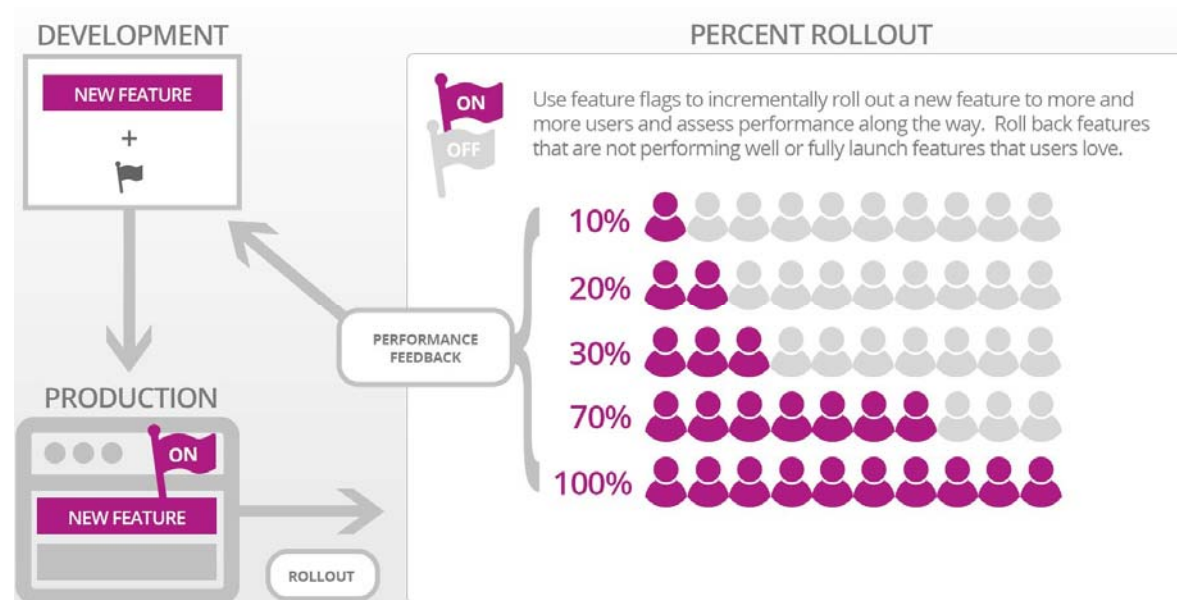
Die Inbetriebnahme beendet die Phase des „Deployments“ im Produktivsystem. Sie ist ein wichtiger Teil der „Ops“-Phase.

4.5 SD, Phase 5: – Incremental Roll-Out („Ops“)

14. Inkrementelles Roll-Out („Canary Release“) führt zu höherer Sicherheit bei Übergängen im Betrieb und für Nutzer.

Heute werden moderne Techniken wie Feature Toggles dafür verwendet, mit denen neue Feat. zu- und auch wieder abgeschaltet werden können.

<https://featureflags.io/2015/12/02/devops-feature-flags-incremental-rollouts>



4.6 SD, Phase 6: Betrieb („Ops“)

15. Der **Betrieb** (Operations, „Ops“) umfasst andauernde Aufgaben während des Betriebs der Software im Produktivsystem mit:

- Überwachung (Monitoring):
 - der Software (läuft stabil,, ...),
 - der Hardware (Ressourcennutzung),
 - der Netzwerke (Sicherheit, Angriffe).
- Steuerung (Start, Stop, Reboot der Software, der Datenbanken, Firewalls, usw.)
- Aufzeichnung und Auswertung des Nutzerverhaltens (für **Feedback an Entwickler**).
- **Customer Support**: Hilfe für Nutzer, administr. Aufgaben (z.B. password reset).
- **BUR**: Backup & Recovery. ...

5 Continuous Integration (CI)

Als **Continuous Integration (CI)** bezeichnet man ein Vorgehen in der Software Entwicklung (in „Dev“), bei der parallel stattfindende (Feature-) Entwicklungen kontinuierlich (z.B. täglich) zusammengeführt werden.

- Ziel ist, dass die Differenzen im Quellcode zwischen den Entwicklungen nicht zu groß werden, denn je größer diese Distanz, desto größer ist der Aufwand für die Zusammenführung (-> merge Konflikte).
- CI ist die Voraussetzung für Continuous Software Delivery.

5.1 Einige Regeln („Best Practices“) für CI

Martin Fowler, Best Practices of Continuous Integration:

- Maintain a Single Source Repository.
- Automate the Build.
- Make Your Build Self-Testing.
- Everyone Commits To the Mainline Every Day.
- Every Commit Should Build the Mainline on an Integration Machine.
- Fix Broken Builds Immediately.
- Keep the Build Fast.
- Test in a Clone of the Production Environment.
- Make it Easy for Anyone to Get the Latest Executable.
- Everyone can see what's happening.
- Automate Deployment.

<https://martinfowler.com/articles/continuousIntegration.html>



https://www.youtube.com/watch?v=aoMfbgF2D_4

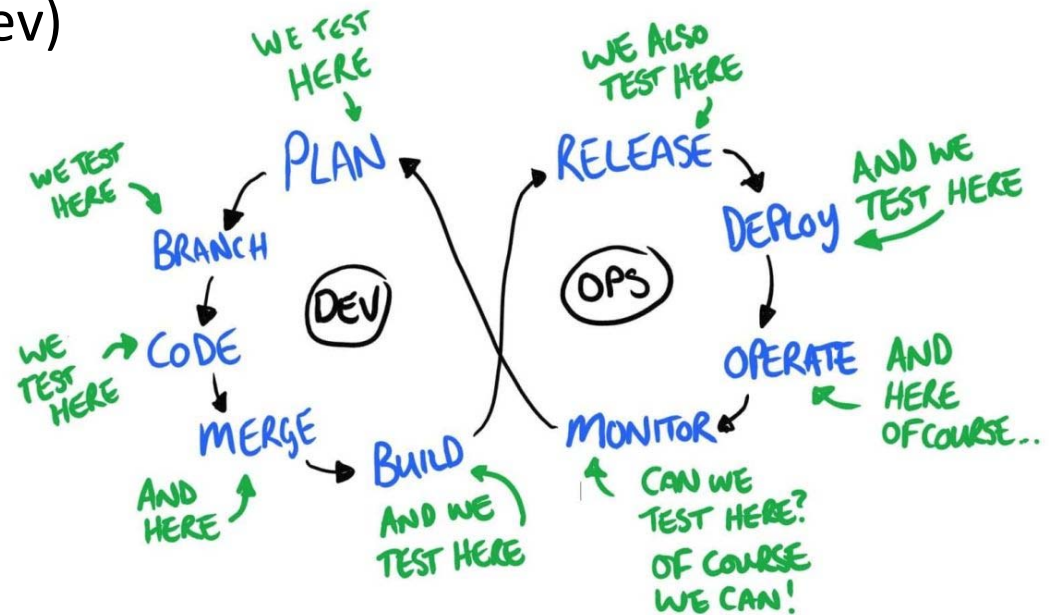
6 Continuous Deployment (CD)

Als **Continuous Deployment (CD)** bezeichnet man ein Vorgehen in der Software Delivery im Übergang zwischen „Dev“ und „Ops“, bei dem ein Stand der Software aus einem Code-Repository (z.B. git) automatisiert in eine Zielumgebung (z.B. Test, Stage, Production) installiert wird.

- Ziel ist, durch automatisierte Ausführung die Zeit für das Deployment zu minimieren (z.B. manuelles Deployment: 3h -> automatisiert: 5min) sowie Fehler bei manuellem Deployment auszuschließen.
- Auch CD ist eine Voraussetzung für Continuous Software Delivery.

7 DevOps

Als **DevOps** bezeichnet man eine Herangehensweise, bei der die Phasen der Software Delivery möglichst reibungsfrei und automatisiert über alle Phasen von Entwicklung (Development, Dev) und Betrieb (Operations, Ops) kontinuierlich ablaufen.



7.1 DevOps Engineer

 [Jobs](#) [Arbeitgeberbewertungen](#) [Gehälter](#) [Interviews](#)  [Anmelden](#)

DevOps Engineer
Babbel

Jobs ▼

Berlin

We are looking for a **DevOps Engineer** for our Data Engineering team (**full-time**) to start immediately in our office in Berlin-Mitte, Germany. In your role, you will work in the Data Engineering team you will evolve our existing data infrastructure and build data services that help build and sell next generation customer features.

You will:

- Manage our AWS infrastructure and services
- Help implement continuous integration processes and automate our deployments
- Secure our cloud environment
- Support software architecture decisions, participate in code reviews

Your profile:

- 3+ years experience in a similar role
- Good knowledge of AWS ecosystem (RDS, Lambda, Kinesis, S3, IAM, VPC)
- Comfortable with Linux environment
- Familiar with at least one infrastructure as code tool (Terraform, CloudFormation, Ansible, Chef or Puppet)
- Proficiency with one (or more) programming language, preferably Python
- Good understanding of CI/CD pipelines

Quelle: <https://www.glassdoor.de>

7.1.1 DevOps Skills (aus dem Bsp. Job-Angebot)

Your will:

- Manage our AWS infrastructure and services.
- Help implement continuous integration processes and automate our deployments.
- Secure our cloud environment.
- Support software architecture decisions, participate in code reviews.

Your profile:

- 3+ years experience in a similar role.
- Good knowledge of AWS ecosystem (RDS, Lambda, Kinesis, S3, IAM, VPC).
- Comfortable with Linux environment.
- Familiar with at least one infrastructure as code tool (Terraform, CloudFormation, Ansible, Chef or Puppet).
- Proficiency with one (or more) programming language, preferably Python.
- Good understanding of CI/CD pipelines.

Zusammenfassung

1 Begriffe (Wiederholung)

2 Was macht ein Software Engineer?

2.1 Software Design Engineer (SDE)

2.2 Software QA Engineer

2.3 Full-Stack Software Engineer

2.4 DevOps Engineer

2.5 Software Architect

3 Software Delivery (SD)

4 Continuous Software Delivery (CSD)

5 Continuous Integration (CI)

6 Continuous Deployment (CD)

7 DevOps